Servidor de Pipelines Concurrente

CI-0117 Programación Paralela y Concurrente

Integrantes:
Gabriel Arce – C5C632
Steve Puerto – C4I552
Bryan Jimenez – C4G330

Link del repositorio: `https://github.com/Ga-ArceMeza18/pipelines_server-c.git`

Introducción

Este servidor tiene la principal función de procesar flujos de trabajos llamados "pipelines" mediante solicitudes de clientes. Cada pipeline realiza tareas definidas en trabajos, etapas y pasos, y pueden contener operaciones que se pueden ejecutar simultáneamente o dependen de algún otro trabajo anterior para poder realizarse. Como varios clientes pueden enviar pipelines o cualquier operación como status, logs, cancel, list o shutdown el servidor debe procesar todas en el orden correcto. Aquí es donde entra el problema de concurrencia que en este proyecto se intenta resolver mediante técnicas de sincronización para que todos los trabajos se completen exitosamente.

La ejecución concurrente introduce desafíos relacionados con la sincronización y la administración de recursos compartidos. Entre estos se encuentran el control de la cantidad de trabajos activos, la actualización de los estados de ejecución, la manipulación de variables y artefactos, y la escritura y lectura simultánea de bitácoras. Una sincronización incorrecta puede provocar condiciones de carrera, inconsistencias en los resultados y comportamientos no deterministas.

Así que el proyecto plantea el diseño e implementación de un servidor de pipelines concurrente capaz de recibir flujos de trabajo en formato YAML, validarlos y ejecutar sus trabajos respetando las dependencias establecidas y los recursos disponibles.

El sistema desarrollado consiste en dos ejecutables principales: un servidor denominado `pipelined` y un cliente denominado `pipectl`. El servidor recibe solicitudes mediante conexiones TCP, procesa definiciones de pipelines en formato YAML, valida su estructura y administra las ejecuciones concurrentes. El cliente permite enviar pipelines, consultar sus estados, visualizar bitácoras, cancelar ejecuciones y solicitar el apagado del servidor.

Las definiciones de los pipelines son estructuradas en etapas, trabajos, dependencias, condiciones de ejecución, variables, pasos y manejo de artefactos. Los pasos pueden consistir en comandos ejecutados mediante procesos hijos o tareas integradas.

El proyecto incluye cuatro tareas integradas: Sleep@1, Cpu@1, WordCount@1 e Histogram@1. Las tareas de procesamiento de datos utilizan OpenMP para distribuir el trabajo entre varios hilos, procurando que sus resultados sean equivalentes a los obtenidos mediante una ejecución

serial.

De manera general, el proyecto se trata en la implementación de mecanismos de sincronización para proteger los recursos compartidos, la realización de pruebas funcionales y concurrentes, y la evaluación experimental del rendimiento bajo diferentes configuraciones.

Diseño de la Solución

La solución fue realizada utilizando un diagrama UML, segmentado por módulos. El engine es el corazón del servidor. Al arrancar con `engine_start`, se crean N hilos de OpenMP, uno por agente (OpenMP Architecture Review Board, 2021). Todos corren `agent_loop`, buscan un trabajo listo, lo ejecutan con `run_job` y cuando terminan actualizan estados y le restan 1 al contador `pending` de los trabajos que dependían de ese. Si algún `pending` llega a 0, ese trabajo ya está listo para que otro agente lo tome.

También tiene submit (agrega un pipeline nuevo y devuelve el id), cancel, status, list y shutdown. Todo lo compartido (la lista de runs y los estados) está protegido por un solo candado global, `g_lock`. Otro módulo es run, que es una ejecución de un pipeline. Guarda el id, el Pipeline, el estado general, la bandera cancel y la carpeta donde van las bitácoras, los espacios de trabajo y los artefactos. Si se manda el mismo YAML dos veces son dos Run distintos.

El Runner ejecuta un trabajo completo. Prepara la carpeta del trabajo, calcula las variables heredadas de los ancestros (`heredar_variables`), corre los pasos en orden (`run_step`), aplica continueOnError, copia los artefactos (`copiar_artefacto` en publish y download) y al final decide el estado del trabajo, ya sea succeeded, partiallySucceeded, failed o canceled. El processRunner lanza un comando como proceso hijo con `posix_spawn`, con la salida redirigida a la bitácora del trabajo. Mientras espera revisa dos cosas: si pidieron cancelar y si se pasó el timeout. Si pasa alguna, mata al grupo de procesos completo (`matar_grupo`) para que no queden nietos vivos. Al final lee las líneas `##setvariable` de la bitácora (`leer_setvariable`) y las mete en las variables del trabajo.

Por otra parte, también está el servidor `pipelined`. Recibe el puerto, la cantidad de agentes, el directorio de trabajo y opcionalmente los hilos por tarea. Al iniciar arranca el engine y se queda aceptando conexiones. Por cada conexión se crea un hilo POSIX desacoplado que lee una línea con la solicitud, responde y cierra el socket, es decir, se atiende una sola solicitud por conexión (Stevens et al., 2004). Aquí se usan pthreads y no OpenMP porque no se sabe cuántos clientes van a llegar ni en qué momento. Las conexiones activas se guardan en una lista protegida por un mutex junto con un contador de hilos, lo cual sirve para el apagado. Cada socket tiene un tiempo límite de lectura de 30 segundos para que un cliente que no escribe no ocupe un hilo para siempre, y se ignora SIGPIPE para que un cliente que se desconecta antes de tiempo no termine el programa (Kerrisk, 2010).

Cuando llega SHUTDOWN se cierra el socket de escucha y la lectura de cada conexión de la lista, así los hilos que esperaban a un cliente salen de inmediato. Luego se suelta el mutex y se llama a `engine_shutdown`, que cancela lo que está corriendo y espera a los agentes. Se

llama sin el mutex porque puede tardar y si se retuviera, los hilos que cierran su conexión no podrían sacarse de la lista. Al final el hilo principal espera a que el contador de hilos llegue a cero antes de liberar los recursos.

El cliente `pipectl` se comunica con el servidor por TCP con mensajes de texto. Envía una línea como LIST, STATUS 1 o CANCEL 1 y el servidor responde con OK, o con ERROR y un mensaje, seguido del contenido si lo hay. En SUBMIT se envía el tamaño del YAML en la misma línea porque el archivo tiene sus propios saltos de línea, entonces con el tamaño el servidor sabe exactamente cuántos bytes leer, con un máximo de 1 MB. Como hay una sola solicitud por conexión, el cierre marca el final de la respuesta. Por defecto `pipectl` se conecta a localhost en el puerto 5000 y termina con código 0 si la respuesta fue OK o 1 si fue ERROR.

Justificación de Sincronización

(1) ¿Cómo se garantiza que nunca haya más trabajos en ejecución que agentes, aun con muchos pipelines simultáneos, y que ningún pipeline espere indefinidamente?

Se garantiza porque el límite está definido por los propios agentes: los agentes son los N hilos de OpenMP y cada hilo corre un solo trabajo a la vez. No hay un contador aparte que pueda quedar mal, así que no importa cuántos pipelines haya. Un pipeline no se puede colar delante de otro que ya estaba esperando, entonces no hay inanición. Los agentes sin nada que hacer esperan con una variable de condición sobre `g_lock` y los despiertan cuando entra un trabajo listo o cuando hay shutdown.

(2) ¿Cómo se consulta el estado (status, list) mientras los trabajos lo modifican, sin leer estados inconsistentes?

Todo cambio de estado se hace con `g_lock` tomado, y status y list también toman `g_lock` para leer. Dentro del candado se arma el texto de respuesta (una copia) y se suelta antes de mandarlo por el socket, así no se retiene el candado mientras se envía por la red. Con eso nunca se lee un estado a medias, por ejemplo un trabajo en running con su run ya en succeeded. Cada cambio de estado guarda también su hora, de ahí sale la duración.

(3) ¿Cómo se evita que las líneas de trabajos paralelos se mezclen o se corten, y cómo se lee una bitácora mientras se escribe (logs `--follow`)?

Las líneas de trabajos paralelos no se mezclan porque cada trabajo tiene su propio archivo de bitácora, así dos trabajos nunca escriben en el mismo archivo. Dentro de un trabajo, las líneas que escribe el engine se arman completas y se envían con un solo `write` sobre un archivo abierto con `O_APPEND`. Con esta bandera el sistema mueve la posición al final y escribe en una sola operación, por lo que ninguna escritura queda en medio de otra y cada línea queda completa (Kerrisk, 2010). En cambio, lo que imprime un script va directo al archivo y el script puede escribir una línea en varias partes. Así que puede haber momentos en que el archivo tenga una línea a medias, lo cual se resuelve al momento de leer.

Para leer mientras se escribe, el lector guarda hasta qué byte envió de cada archivo. Con `--follow`, cada 200 ms lee lo nuevo desde esa posición y envía solo hasta el último salto de línea, lo demás se queda para la siguiente vuelta, de esta forma el cliente nunca recibe una línea cortada. Cuando el engine indica que la ejecución terminó, hace una última lectura y cierra la conexión. El lector no necesita un candado propio porque solo lee y el estado lo consulta con funciones del engine que ya toman `g_lock`.

(4) ¿Qué ocurre si dos trabajos paralelos definen la misma variable y un tercero depende de ambos? La regla debe ser determinista.

Cada trabajo tiene su propio `out_vars` y solo lo escribe él mismo, así que no hay dos hilos escribiendo la misma lista. Un trabajo hereda las variables hasta que todos sus ancestros ya terminaron (`pending` = 0), así que ya no cambian.

Para el caso de dos trabajos paralelos que definen la misma variable y un tercero depende de ambos, la regla es que "gana" el trabajo que aparece después en el YAML. Se recorren los ancestros en el orden del YAML y el último reemplaza al anterior. No depende de quién terminó primero, entonces es determinista. La prioridad, de menor a mayor, es: variables del pipeline, las heredadas y las que el propio trabajo define con `##setvariable`.

(5) ¿Qué ocurre si dos trabajos publican un artefacto con el mismo nombre, o si un trabajo descarga uno que todavía no existe?

Si dos trabajos publican el mismo nombre, el Validator lo rechaza antes de ejecutar (`check_artefactos`), entonces en ejecución nunca ocurre. Si se intenta descargar uno que no existe, también se valida antes, y se rechaza si ningún ancestro lo publica. Como el que publica es ancestro, ya terminó antes de que el que descarga empiece, así que el artefacto ya está ahí.

Para publicar, se copia a un nombre temporal dentro de la carpeta de artefactos del run y después se hace `rename`, que es atómico, así nadie ve un artefacto que no se ha copiado totalmente. Si aun así falta en ejecución, el paso download falla con un mensaje claro.

(6) ¿Cómo se detienen los procesos hijos y los trabajos pendientes sin dejar recursos sin liberar ni estados a medias?

El comando cancel pone `run->cancel = true` con `g_lock` tomado. El agente que está dentro de `run_process` revisa cada cierto tiempo corto si cancel está en true o si se pasó el deadline del timeout. Si pasa algo de eso, manda SIGTERM a todo el grupo de procesos (`kill(-pgid)`), espera un momento, y si siguen vivos manda SIGKILL. Después hace `waitpid` para recoger al hijo (no quedan zombies) y cierra los descriptores.

Los trabajos pendientes no se ejecutan: los agentes los ven con cancel y los marcan como skipped. Los que estaban corriendo quedan canceled. El run pasa a canceled hasta que ya no queda ningún trabajo en running, y todos esos cambios se hacen bajo `g_lock`, así no quedan estados a medias. Un timeout hace que el trabajo termine en failed.

(7) ¿Cómo se combinan los resultados parciales de cada hilo sin carreras y con un resultado idéntico al serial?

Cada hilo usa sus propios contadores privados y al final se combinan en orden fijo de hilo,

sin escribir a la vez en la misma estructura. Como son conteos enteros, el resultado es igual al serial con cualquier número de hilos. Para el top de WordCount se ordena por frecuencia y, en empate, alfabéticamente, para que también sea determinista.

(8) ¿Qué implicaciones tiene crear procesos (fork/exec o posix_spawn) desde un programa con varios hilos, y cómo las manejó el grupo?

Hacer `fork` en un programa con varios hilos es peligroso, porque el hijo solo copia el hilo que llamó a `fork`, pero puede heredar candados tomados por otros hilos, y entre `fork` y `exec` solo es seguro llamar funciones async-signal-safe (Kerrisk, 2010). Por eso `posix_spawn` es útil, pues evita ese problema (The Open Group, 2018). El hijo se crea en su propio grupo (`POSIX_SPAWN_SETPGROUP`) para poder matar al grupo completo. Además, los descriptores del servidor (socket, bitácoras de otros trabajos) se abren con `O_CLOEXEC` para que el hijo no los herede.

(9) ¿Cómo interactúan los hilos de OpenMP de una tarea con los de otros trabajos que corren al mismo tiempo? ¿Cómo se evita la sobresuscripción del procesador?

Los agentes son una región paralela de OpenMP, y las tareas de datos abren otra región dentro, o sea hay anidamiento. Si N agentes pidieran cada uno h hilos, habría $N \cdot h$ hilos peleando por los núcleos (sobresuscripción). Para que el anidamiento funcione se activa con `omp_set_max_active_levels(2)`, y para evitar la sobresuscripción, si no se indica la cantidad de hilos por tarea, el engine reparte los núcleos entre los agentes, es decir, cada tarea recibe núcleos / agentes hilos, con un mínimo de 1 (OpenMP Architecture Review Board, 2021). Así la suma de hilos activos no pasa aproximadamente de la cantidad de núcleos.

Batería de Pruebas

Las pruebas se encuentran en la carpeta `tests` del repositorio y se ejecutan todas con `make test`, que compila cada una y la corre en orden. Cada prueba muestra [OK] o [FAIL] por cada verificación y al final un resumen con la cantidad de pruebas aprobadas. Se dividen en dos tipos. Las pruebas unitarias están escritas en C y revisan cada módulo por separado sin levantar el servidor, por ejemplo el parser, las tareas integradas o las funciones de red, así que si una falla se sabe exactamente en qué parte está el problema. Por otro lado, la prueba de integración y concurrencia es un script que levanta el servidor real y lo usa por medio de `pipectl`, de esta forma se prueba que todos los módulos funcionen juntos y que el servidor responda bien cuando recibe varias solicitudes al mismo tiempo.

La prueba del parser utiliza los archivos YAML de la carpeta `pipelines`, tres válidos y cinco inválidos. Los válidos tienen un pipeline básico con variables, uno con dependencias entre trabajos y uno con condiciones. Los inválidos tienen un error de sintaxis, una clave que no existe, una dependencia a un trabajo que no existe, un ciclo entre trabajos y un nombre repetido. Para cada archivo se revisa que el parser lo acepte o lo rechace según corresponda, y si el resultado no es el esperado se muestran los errores encontrados con su línea.

Para las tareas integradas hay dos pruebas. La de tareas simples revisa que Sleep@1 espere

el tiempo pedido, que Sleep@1 y Cpu@1 rechacen entradas inválidas y que se detengan al cancelar o al vencer el tiempo límite. La de tareas de datos ejecuta WordCount@1 sobre un texto con espacios, tabulaciones y líneas vacías, y Histogram@1 sobre una lista de números con valores negativos y repetidos, y compara la salida con el resultado que se sabe correcto. En las tres tareas que usan OpenMP, Cpu@1, WordCount@1 e Histogram@1, se repite la ejecución con 1, 2, 4 y 8 hilos y se revisa que el resultado sea exactamente el mismo. También se comprueba que fallen si el archivo de entrada no existe.

La prueba de `net_util` revisa las funciones de red que comparten el servidor y el cliente, por ejemplo, que un cliente se conecte, que se envíe 1 MB sin cambios, que las líneas se separen bien y que al leer la línea de SUBMIT no se consuman los bytes del YAML que vienen después.

La prueba de determinismo usa el engine directamente sin el servidor con 16 agentes. Primero ejecuta 10 veces un pipeline donde dos trabajos escriben la misma variable y un tercer trabajo depende de ambos, y revisa que el tercero reciba siempre el mismo valor, el del segundo trabajo según el orden del YAML, sin importar cuál de los dos terminó primero. Después envía un pipeline con 512 trabajos independientes y comprueba que todos terminen en succeeded, que el contador de trabajos terminados llegue exactamente a 512 y que nunca haya más trabajos corriendo que agentes. Esta última parte es la que busca la condición de carrera, porque muchos agentes terminan trabajos al mismo tiempo y actualizan los mismos contadores.

Para la integración y la concurrencia se hizo `test_protocol.sh`, que arranca el servidor real con 2 agentes y lo usa por medio de `pipectl`, igual que lo haría un usuario, así cada solicitud pasa por la red, el parser, el engine y las bitácoras al mismo tiempo. Con esto se revisa que un pipeline válido termine en succeeded, que los inválidos se rechacen, que status, logs y cancel respondan bien y que los errores no afecten al servidor. Además, varias de estas verificaciones ponen al servidor a atender muchas cosas a la vez, que es donde aparecen los problemas de sincronización. Por ejemplo, se envían tres pipelines de tres trabajos y mientras corren se cuenta cada 0.1 segundos cuántos trabajos hay en running, el máximo nunca pasó de 2, que es la cantidad de agentes, y al correrla con un límite falso de 1 la prueba falló, lo que confirma que sí detecta el error. También se conectan 20 clientes al mismo tiempo, se sigue una bitácora con logs `--follow` mientras el trabajo escribe sin que lleguen líneas cortadas y se envía SHUTDOWN con una ejecución en curso para comprobar que el servidor la cancela y termina de forma ordenada.

Métricas

Metodología

Las mediciones se hicieron con el programa `tools/bench.c`, que levanta el servidor, envía los pipelines de cada experimento y toma el tiempo hasta que la ejecución termina. Cada configuración se repitió 5 veces y se utilizó la mediana, ya que es menos sensible a una medición atípica que el promedio. Los experimentos son los siguientes:

- **Pipeline ancha:** 8 trabajos independientes con Cpu@1, variando la cantidad de agentes.
- **Pipeline profunda:** 8 trabajos con Cpu@1 en cadena, donde cada uno depende del anterior, variando los agentes.
- **Pipelines simultáneas:** 4 pipelines anchas enviadas al mismo tiempo, variando los agentes.
- **Cpu@1, WordCount@1 e Histogram@1:** un solo trabajo con 1 agente, variando los hilos de OpenMP de la tarea. En WordCount e Histogram se mide solo el tiempo del paso de la tarea.

En los tres primeros experimentos cada tarea usa un solo hilo, así que lo único que cambia es el paralelismo entre trabajos que dan los agentes. Con la mediana T_p de cada configuración se calculó el speedup $S_p = T_1/T_p$ y la eficiencia $E_p = S_p/p$, donde p es la cantidad de agentes o de hilos. Para estimar la fracción paralelizable f se despejó la ley de Amdahl (Amdahl, 1967)

$$S_p = \frac{1}{(1-f) + f/p} \qquad \Longrightarrow \qquad f = \frac{1 - 1/S_p}{1 - 1/p}$$

para cada valor de $p > 1$ y se tomó la mediana de esas estimaciones, limitada al intervalo $[0, 1]$.

Ambientes

Las pruebas se ejecutaron en tres ambientes con hardware distinto. En el ambiente 2 se probó además con 12 agentes y se ejecutó el experimento de Cpu@1.

Ambiente	Procesador	Sistema
1	Intel Core i3-1125G4, 4 núcleos y 8 hilos	Linux
2	Intel Core i7, 8 núcleos	Linux en máquina virtual
3	Intel Core i3-6006U, 2 núcleos y 4 hilos	Ubuntu 26.04, GCC 15.2

Tabla 1: Ambientes de prueba.

Resultados

	Ambiente 1		Ambiente 2		Ambiente 3	
Agentes	T_p (s)	S_p	T_p (s)	S_p	T_p (s)	S_p
1	10.489	1.000	10.135	1.000	14.574	1.000
2	5.938	1.766	5.267	1.924	8.155	1.787
4	3.950	2.656	3.816	2.656	7.748	1.881
8	2.516	4.169	2.724	3.721	7.557	1.929
12	–	–	2.647	3.829	–	–
f	0.868		0.834		0.625	

Tabla 2: Pipeline ancha: 8 trabajos independientes.

	Ambiente 1		Ambiente 2		Ambiente 3	
Agentes	T_p (s)	S_p	T_p (s)	S_p	T_p (s)	S_p
1	9.120	1.000	10.085	1.000	14.539	1.000
2	9.081	1.004	8.908	1.132	14.766	0.985
4	8.892	1.026	9.005	1.120	14.697	0.989
8	8.860	1.029	8.612	1.171	14.749	0.986
12	–	–	9.123	1.105	–	–
f	0.033		0.155		0.000	

Tabla 3: Pipeline profunda: 8 trabajos en cadena.

	Ambiente 1		Ambiente 2		Ambiente 3	
Agentes	T_p (s)	S_p	T_p (s)	S_p	T_p (s)	S_p
1	37.122	1.000	34.894	1.000	59.825	1.000
2	21.228	1.749	20.365	1.713	34.686	1.725
4	14.093	2.634	13.655	2.555	30.430	1.966
8	11.352	3.270	8.856	3.940	28.928	2.068
12	–	–	7.551	4.621	–	–
f	0.827		0.843		0.655	

Tabla 4: Pipelines simultáneas: 4 pipelines de 8 trabajos.

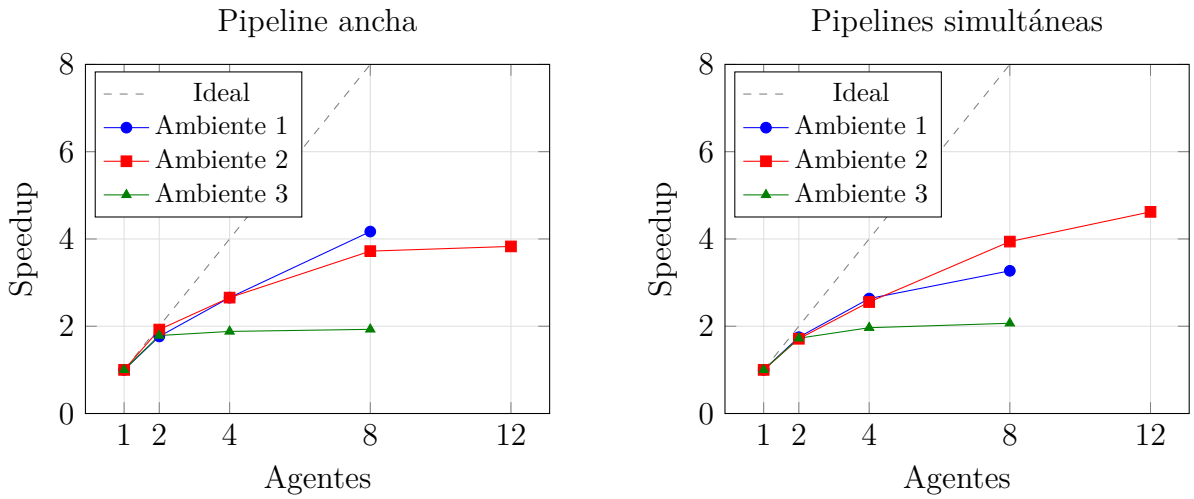


Figura 1: Speedup según la cantidad de agentes.

	Ambiente 1		Ambiente 2		Ambiente 3	
Hilos	T_p (s)	S_p	T_p (s)	S_p	T_p (s)	S_p
1	0.18	1.000	0.94	1.000	0.29	1.000
2	0.13	1.385	0.82	1.146	0.21	1.381
4	0.09	2.000	0.68	1.382	0.22	1.318
8	0.07	2.571	0.80	1.175	0.16	1.812
12	–	–	0.90	1.044	–	–
f	0.667		0.213		0.512	

Tabla 5: WordCount@1 según los hilos de OpenMP.

	Ambiente 1		Ambiente 2		Ambiente 3	
Hilos	T_p (s)	S_p	T_p (s)	S_p	T_p (s)	S_p
1	0.51	1.000	8.07	1.000	1.37	1.000
2	0.62	0.823	7.07	1.141	1.43	0.958
4	0.64	0.797	7.03	1.148	1.42	0.965
8	0.46	1.109	7.59	1.063	1.71	0.801
12	–	–	8.21	0.983	–	–
f	0.000		0.120		0.000	

Tabla 6: Histogram@1 según los hilos de OpenMP.

Hilos	T_p (s)	S_p	E_p (%)
1	1.341	1.000	100.00
2	0.811	1.654	82.72
4	0.794	1.689	42.22
8	0.931	1.441	18.01
12	1.904	0.704	5.87
f		0.447	

Tabla 7: Cpu@1 según los hilos de OpenMP (ambiente 2).

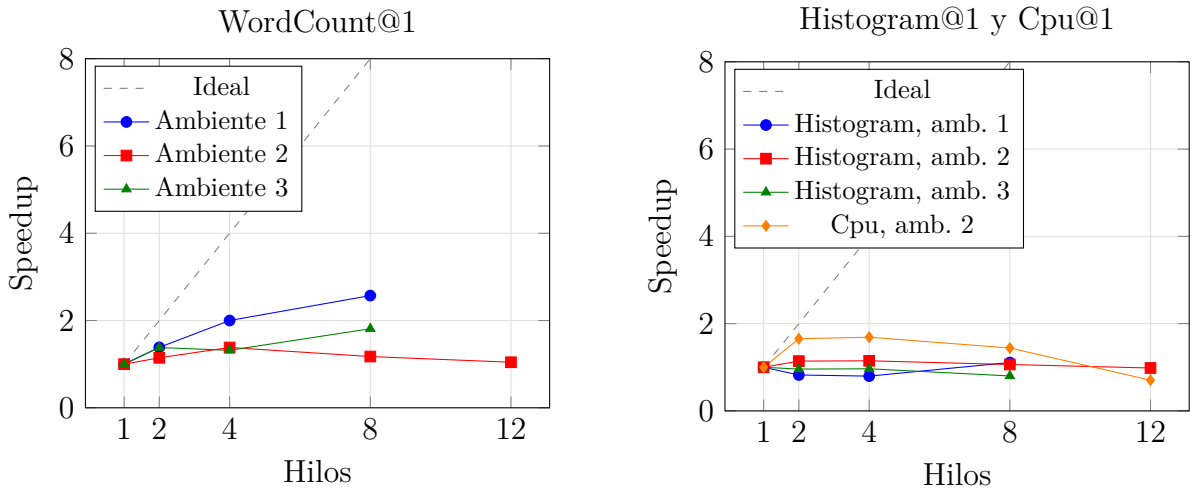


Figura 2: Speedup según los hilos de OpenMP de la tarea.

Análisis de Resultados

En la pipeline ancha los 8 trabajos no dependen entre sí, así que es el caso donde más se nota el aporte de los agentes. En los ambientes 1 y 2 el speedup llega a 4.17 y 3.72 con 8 agentes y la fracción paralelizable estimada es de 87 % y 83 %. En cambio, en el ambiente 3 el speedup casi no pasa de 1.9 a partir de 2 agentes. Esto es debido a que ese procesador tiene solo 2 núcleos físicos, entonces con más de dos trabajos de Cpu@1 al mismo tiempo los agentes se reparten los mismos núcleos y el tiempo total ya no baja. Los hilos lógicos de Hyper-Threading ayudan poco en una tarea que solo hace cálculos. Por el mismo motivo, en el ambiente 2 pasar de 8 a 12 agentes casi no mejora el resultado, ya que hay más agentes que núcleos.

En la pipeline profunda el speedup se mantiene cerca de 1 en los tres ambientes sin importar la cantidad de agentes. Este resultado es el esperado, porque cada trabajo depende del anterior y nunca hay dos trabajos listos al mismo tiempo, es decir, los agentes adicionales se quedan esperando. Por eso la fracción paralelizable sale cercana a 0, y la pequeña mejora del ambiente 2 se puede atribuir a la variación entre mediciones. Este experimento muestra

que el paralelismo que se puede obtener depende de la forma del pipeline y no solo de los recursos disponibles.

Con pipelines simultáneas el comportamiento es parecido a la pipeline ancha, ya que hay 32 trabajos independientes repartidos en 4 ejecuciones. El ambiente 2 es el que mejor escala y llega a un speedup de 4.62 con 12 agentes. Además, esto confirma que el engine reparte los agentes entre varias ejecuciones sin que una deje esperando a las demás. El ambiente 3 vuelve a quedarse cerca de 2 por la cantidad de núcleos.

En las tareas de datos los tiempos son mucho más cortos, por lo que el costo de crear los hilos y la variación entre mediciones pesan más. WordCount@1 escala en el ambiente 1 hasta 2.57 con 8 hilos, pero en los otros dos ambientes la mejora es menor. Histogram@1 prácticamente no mejora con más hilos y en algunos casos empeora, por lo que la fracción paralelizable estimada sale cercana a 0. Esto indica que en esta tarea domina una parte serial, probablemente la lectura y conversión del archivo de entrada, y que la parte que se reparte entre los hilos es pequeña en comparación. En el ambiente 2 los tiempos de ambas tareas son bastante más altos que en los otros, lo cual puede deberse a que la máquina virtual hace más lenta la lectura de archivos.

Por último, en Cpu@1 el speedup mejora hasta 2 y 4 hilos, pero con 8 hilos baja y con 12 el tiempo es incluso mayor que con 1 hilo. Esto ocurre porque al pedir más hilos que núcleos disponibles se produce sobresuscripción, y los hilos compiten por el procesador. Este resultado justifica que el engine, si no se indica lo contrario, reparta los núcleos entre los agentes en lugar de dejar que cada tarea use todos.

En general, los resultados muestran que el hardware del ambiente afecta directamente cuánto se puede acelerar el servidor, siendo la cantidad de núcleos físicos el factor que más limita el speedup.

Conclusiones

Un servidor de pipelines concurrente tiene retos en dos niveles. Por un lado está la ejecución, donde varios agentes toman trabajos al mismo tiempo, actualizan estados y contadores, heredan variables y publican artefactos, por lo que todo lo compartido debe protegerse para que los resultados sean correctos y deterministas. Por otro lado está la atención de clientes, donde el servidor debe responder a muchas solicitudes a la vez, permitir consultar el estado y seguir las bitácoras mientras los trabajos corren, y apagarse sin dejar procesos ni recursos sin liberar. Para esto se combinaron hilos POSIX en la red, OpenMP en los agentes y en las tareas, un candado global en el engine y un mutex con variable de condición en el servidor.

La batería de pruebas permitió revisar cada módulo por separado y también el sistema completo funcionando de forma concurrente. Las pruebas de integración confirmaron que nunca hay más trabajos corriendo que agentes, que las bitácoras se leen sin líneas cortadas y que el apagado es ordenado, mientras que la prueba de determinismo comprobó que la regla de herencia de variables y la contabilidad de trabajos se mantienen correctas con muchos agentes.

Respecto a las métricas, el speedup depende tanto de la forma del pipeline como del hardware. Un pipeline con trabajos independientes aprovecha bien los agentes, mientras que una cadena de dependencias no mejora sin importar cuántos agentes haya. Además, el número de núcleos físicos de cada ambiente limita la aceleración, y usar más hilos que núcleos puede empeorar el rendimiento, lo que muestra la importancia de evitar la sobresuscripción.

Referencias

Amdahl, G. M. (1967). Validity of the single processor approach to achieving large scale computing capabilities. En *Proceedings of the April 18-20, 1967, Spring Joint Computer Conference* (pp. 483–485). Association for Computing Machinery. `https://doi.org/10.1145/1465482.1465560`

Kerrisk, M. (2010). *The Linux programming interface: A Linux and UNIX system programming handbook*. No Starch Press.

OpenMP Architecture Review Board. (2021). *OpenMP application programming interface* (Versión 5.2). `https://www.openmp.org/specifications/`

Stevens, W. R., Fenner, B. y Rudoff, A. M. (2004). *UNIX network programming: Vol. 1. The sockets networking API* (3.ª ed.). Addison-Wesley.

The Open Group. (2018). *The Open Group base specifications issue 7* (IEEE Std 1003.1-2017). `https://pubs.opengroup.org/onlinepubs/9699919799/`